

# COSC 3337 Final Project

Design and Build a Database Application

St. Edward's University · Fall 2026

## Overview

In teams of 3–4, you will design a relational database for a domain of your choosing and build a working application on top of it. By the end of the semester, a user who knows nothing about databases should be able to sit down at your application and complete its core tasks without ever knowing SQL exists.

The project follows a standard database design methodology in three stages: you propose an idea, you design the database, and you build and present the finished application.

| Deliverable                   | Due                               | Weight |
|-------------------------------|-----------------------------------|--------|
| Checkpoint 1: Team & proposal | Sunday, October 11, 11:59 PM      | 10%    |
| Checkpoint 2: Database design | Sunday, November 8, 11:59 PM      | 25%    |
| Final application & report    | Monday, December 7, 11:59 PM      | 65%    |
| Presentation & live demo      | Tuesday, December 8, 3:45–5:45 PM |        |

All deliverables are submitted as PDFs on Canvas as a group. Peer evaluations are due with the final submission. For your convenience, you are provided with LaTeX templates for each checkpoint and the final report, but you are not required to use them, as long as you submit a professional document. There is no bonus associated with using LaTeX for the final report.

## 1 Teams

- Teams have 3 or 4 members. Submit your roster as part of Checkpoint 1 (Sunday, October 11, 11:59 PM). Students without a team by then will be assigned to one.
- Everyone on a team receives the same grade by default. Each member submits a confidential peer evaluation at the final checkpoint. If evaluations show that someone did substantially less than their share, that person's grade may be adjusted. I will also check the Git repository to see who committed code.
- Good teamwork is not just dividing the work. Every member should be able to explain the schema and answer questions about any part of the application during the presentation.

## 2 Choosing a Domain

You choose what your application does. Pick something you would actually want to use or can imagine someone using: a course registration system with waitlists, a marketplace for used textbooks, an appointment booker for a salon, a rec-league stats tracker, a shelter adoption system, and so on. I encourage you to use this project as an opportunity to build your developer portfolio. A well-designed, working application in a real domain is a strong addition to your resume.

A good domain has:

- Several kinds of things (entities) that relate to each other in interesting ways, including at least one many-to-many relationship.

- Actions that change data, not just browsing. Something has to be created, updated, reserved, purchased, or cancelled.
- Enough data to make queries meaningful: real public data, scraped data, or realistic generated data.

Avoid domains that are just a single list with a search box, or so large that your schema will not fit in two dozen tables.

### 3 Suggested Data Sources and Tools

#### Your tech stack

Every student has a course VM with MySQL already installed, and your team is welcome to build on it, but you do not have to. You may use a different relational database or hosting setup if you want to try something new, as long as:

- it is a **relational** database queried with SQL, and it supports every SQL feature the project requires (outer joins, GROUP BY/HAVING, subqueries, set operations, and views);
- your queries are written in SQL by your team, not generated by an ORM. You may use an ORM or query builder for simple inserts and lookups, but the 8 required queries and your view must be hand-written SQL;
- you tell me which stack you chose in your Checkpoint 1 proposal;
- I can run your application or see it demonstrated at the final checkpoint.

**Note:** If you use the course VM, the VM will be deleted at the end of the semester. You are responsible for backing up your work and moving it to a permanent location if you want to keep it.

#### Where to find data

- **City of Austin Open Data Portal** (<https://data.austintexas.gov>): restaurant inspections, permits, parks, 311 requests, and more.
- **Data.gov** (<https://data.gov>): U.S. federal datasets on nearly every topic.
- **Kaggle Datasets** (<https://www.kaggle.com/datasets>): thousands of community-posted CSVs; check each dataset's license.
- **IMDb Non-Commercial Datasets** (<https://developer.imdb.com/non-commercial-datasets/>): movies, people, and ratings.
- **Open Library** (<https://openlibrary.org/developers>): books, authors, and editions through downloads or an API.
- **MusicBrainz** ([https://musicbrainz.org/doc/MusicBrainz\\_Database](https://musicbrainz.org/doc/MusicBrainz_Database)): artists, releases, and recordings.
- **Transit GTFS feeds**, such as CapMetro's, published through the Austin portal and <https://mobilitydatabase.org>: routes, stops, and schedules.

**Generating realistic data.** If you public dataset is available, you may generate the data you need. **Mockaroo** (<https://www.mockaroo.com>) produces CSV or SQL inserts from a column spec, and the Python library **Faker** (<https://faker.readthedocs.io>) generates names, addresses, dates, and more in your own load script. You may also combine a real dataset with generated users or transactions.

## Tools

- **ER diagrams:** draw.io (<https://app.diagrams.net>) or any tool that supports the notation from class. dbdiagram.io (<https://dbdiagram.io>) is handy for drawing the relational schema.
- **Database clients:** MySQL Workbench (<https://dev.mysql.com/downloads/workbench/>) or DBeaver (<https://dbeaver.io>) for browsing tables and testing queries.
- **Other relational databases:** PostgreSQL (<https://www.postgresql.org>) or SQLite (<https://www.sqlite.org>). Hosted options such as Supabase (<https://supabase.com>) run PostgreSQL for you.
- **Web frameworks:** Flask or Django (Python), Express (Node.js), plain PHP, or Spring Boot (Java). Each has a driver that supports parameterized queries.
- **Deployment (extra credit):** services such as Render, Railway, or PythonAnywhere can host a small web app and database. Free tiers change often, so check current limits before you commit to one.

## 4 Checkpoint 1: Team and Proposal Due Sunday, October 11, 11:59 PM

Submit a PDF containing:

1. **Team roster.** The names of all team members.
2. **Domain and users.** What the application does, who uses it, and what problem it solves. If the application has more than one kind of user (e.g., buyers and sellers, students and administrators), describe each.
3. **Ten questions.** Ten questions your database must be able to answer, written in plain English from the user's point of view. Example: "*Which items has this seller listed that have received no bids in the past week?*" At least four should require combining information from more than one kind of entity, and at least two should involve counts, totals, or averages. These questions will become the queries in your final application.
4. **Data source.** Where your data will come from and roughly how many records you expect.
5. **Entities.** A rough list of the main entities and how they relate. This does not need to be a formal ER diagram yet.
6. **Tech stack.** The database and framework you plan to use, and whether you will run on a course VM.
7. **Git repository.** A link to your team's Git repository. You may choose to make it private or public, but I must be invited as a collaborator so I can view your work. (Username: isabelcachola). If you choose to make it public, be extra careful not to commit any real passwords, keys, or other sensitive information.

**Proposal meeting.** Within two weeks of the Checkpoint 1 deadline, your team must meet with me to discuss the proposal. Sign up for a time slot once you submit. If you do not meet with me, your Checkpoint 1 grade will be a zero.

**Grading (10 points)**

|                                                                                 |               |
|---------------------------------------------------------------------------------|---------------|
| Team roster                                                                     | 1             |
| Clear domain, users, and problem statement                                      | 2             |
| Ten questions: specific, answerable, meet the join and aggregation requirements | 4             |
| Realistic data source                                                           | 1             |
| Entity list shows enough complexity for the project                             | 1             |
| Git repository                                                                  | 1             |
| Meeting with instructor                                                         | 0 (pass/fail) |

**5 Checkpoint 2: Database Design      Due Sunday, November 8, 11:59 PM**

Submit a PDF containing:

1. **ER diagram.** A complete ER diagram using the notation from class. It must include:
  - at least 6 entity sets;
  - at least one many-to-many relationship set;
  - at least one weak entity set;
  - cardinality and participation constraints on every relationship.
2. **Relational schema.** The schema produced from your ER diagram, with every primary key and foreign key marked. For the weak entity and the many-to-many relationship, briefly explain how you translated each into tables.
3. **Updated questions.** Your ten questions, revised if your design changed them.

You will receive feedback on your design. You may change the design afterward; document the changes in your final report.

**Grading (25 points)**

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| ER diagram meets all structural requirements and uses correct notation        | 10 |
| ER diagram accurately models the proposed domain                              | 5  |
| Correct translation to relational schema, keys marked, translations explained | 8  |
| Updated questions                                                             | 2  |

**6 Final Application****6.1 Database implementation**

Your database may run on a course VM or on any other relational DBMS your team chooses (see Section 3). If you use a database server, create a separate user for each teammate and grant each the privileges they need. The database must include:

- **Data:** at least a few hundred rows in total across your tables, loaded by a script included in your repository.
- **At least 8 non-trivial queries** answering your proposal questions. Across them, you must use at least one of each:
  - an outer join;
  - GROUP BY with HAVING;

- a subquery (nested in **WHERE**, **FROM**, or **SELECT**);
- a set operation (**UNION**, **INTERSECT**, or **EXCEPT**).
- **At least one view**, used by the application.

## 6.2 Application requirements

Your application must look and behave like a real product. **Users interact only through forms, buttons, dropdowns, search boxes, and clickable results. They never type SQL, see table or column names, or read raw database errors.** The less your application looks like it is talking to a relational database, the better.

- **No query box.** A text area that runs arbitrary SQL, or a page of buttons labeled “Query 1” through “Query 8,” does not satisfy this requirement.
- **Queries sit behind user actions.** Each of your required queries is triggered by something a user would naturally do, such as “Show my overdue items” or filtering listings by price range.
- **Users can change data.** The application supports creating, updating, and deleting records through the interface.
- **Parameterized queries only.** All user input is passed as query parameters, never concatenated into SQL strings.
- **Friendly errors.** Constraint violations (a duplicate username, a double-booked time slot) produce a plain-language message, not a raw database error.
- **Readable output.** Results are formatted for your domain, with meaningful labels and links to detail pages rather than raw result grids.

Any language and framework is fine (Flask, Django, Node/Express, PHP, Java).

## 6.3 Final report

Submit a PDF report containing:

1. **Overview:** what the application does and who it is for.
2. **Final ER diagram and relational schema**, with a list of changes since Checkpoint 2 and why you made them.
3. **Query map:** a table listing each required query, the plain-English question it answers, the screen and user action that runs it, and which required SQL feature(s) it demonstrates. Include the SQL in an appendix.
4. **Views:** what each view does and where the application uses it.
5. **Screenshots** of the main screens.
6. **Contributions:** what each team member did.
7. **Repository link and setup instructions** sufficient for someone to run your application.

**Extra credit (5 points).** Include a link to your application deployed on the web, where anyone can use it.

## 6.4 Presentation

Each team gives a 15-minute presentation (Tuesday, December 8, 3:45–5:45 PM, during the final exam slot) followed by questions. Include:

- an overview of the domain and your ER diagram (about 5 minutes);
- a live demo of the application, showing several queries and a friendly error message (about 9 minutes);
- one thing you would change if you started over (about 1 minute).

Every team member must speak. Have screenshots ready in case your demo is unreachable.

### Final grading (65 points)

#### Database implementation (18)

|                                                            |    |
|------------------------------------------------------------|----|
| Database set up and data loaded by script                  | 4  |
| Eight queries, correct, covering all required SQL features | 10 |
| View used meaningfully by the application                  | 4  |

#### Application (27)

|                                                                  |    |
|------------------------------------------------------------------|----|
| Users never write or see SQL; queries sit behind natural actions | 10 |
| Create, update, and delete work through the interface            | 6  |
| Parameterized queries throughout                                 | 5  |
| Friendly errors and readable output                              | 6  |

#### Report (10)

|                                                                 |    |
|-----------------------------------------------------------------|----|
| Complete, accurate, query map correct, design changes explained | 10 |
|-----------------------------------------------------------------|----|

#### Presentation (10)

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| Clear explanation, working live demo, every member speaks and answers questions | 10 |
|---------------------------------------------------------------------------------|----|

#### Extra credit (up to 5)

|                                                        |    |
|--------------------------------------------------------|----|
| Application deployed and publicly reachable on the web | +5 |
|--------------------------------------------------------|----|

**The standard to aim for:** a user unfamiliar with databases could complete your application's core tasks without knowing SQL exists.

## Academic Integrity

You may use libraries, frameworks, and public datasets, and you must cite them in your report. You may consult tutorials and documentation. The design, schema, queries, and application must be your team's own work. If you use AI tools, follow the course AI policy and disclose how you used them in your report. Every team member must be able to explain any part of the submission.